EDA

Exploratory Data Analysis

A complete beginner-to-pro guide

with a real Titanic worked example + Python code

Process | Charts | Real data | Code

Swipe to learn →

What is EDA?

The first thing you do with any dataset

Looking at your data from every angle
BEFORE building any model.

Understand structure, types, meaning

Clean missing, duplicates, errors

Discover patterns & relationships

Decide features & the right model

"Look at the data first." – John Tukey

The 8-Step EDA Process

1. Define the question & context
2. Load & inspect (shape, dtypes, head)
3. Data quality: missing, duplicates
4. Clean: impute, drop, fix types
5. Univariate: one variable at a time
6. Bivariate: feature vs target
7. Multivariate: interactions, correlation
8. Insights & feature engineering

Steps by Data Type

What to check + which chart to use

Numerical

Check: mean, median, skew, outliers

Chart: Histogram, KDE, Boxplot

Categorical

Check: counts, mode, rare labels

Chart: Bar / count plot

Num vs Cat

Check: group mean, t-test / ANOVA

Chart: Box / Violin plot

Time series

Check: trend, seasonality, gaps

Chart: Line plot, ACF, decomposition

Text

Check: length, word freq, n-grams

Chart: Word bars, word cloud

Geospatial

Check: region counts, clusters

Chart: Maps, choropleth

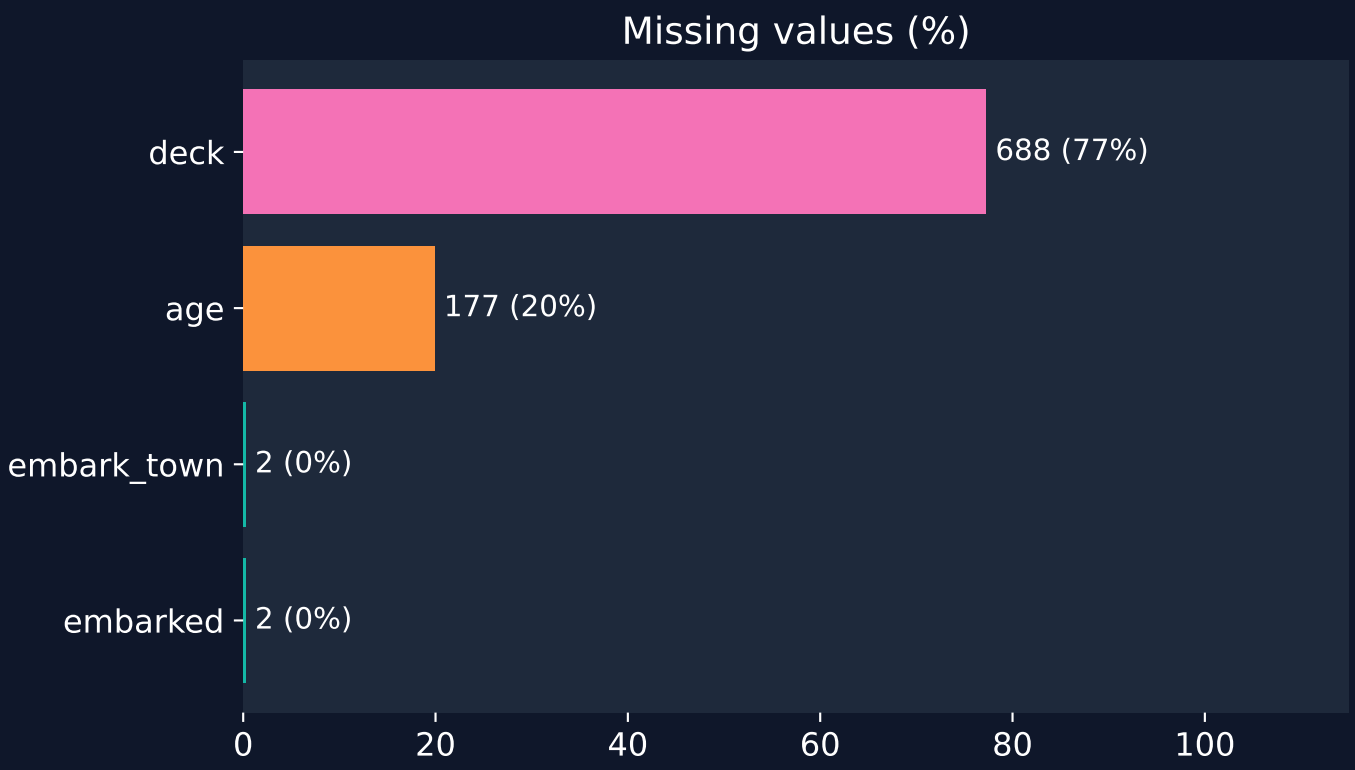
Chart Cheat Sheet

Pick the chart by the question

1 numeric	Histogram • KDE • Boxplot
1 categorical	Bar / Count plot
2 numeric	Scatter • Hexbin • Regplot
Numeric vs category	Boxplot • Violin • Bar of means
2 categorical	Grouped bar • Crosstab heatmap
Many numeric	Correlation heatmap • Pairplot
Missing data	Missing-value heatmap

Example: Titanic Dataset

891 passengers • 15 columns • target = survived



Drop `deck` (77% missing)

Fill `embarked` with mode

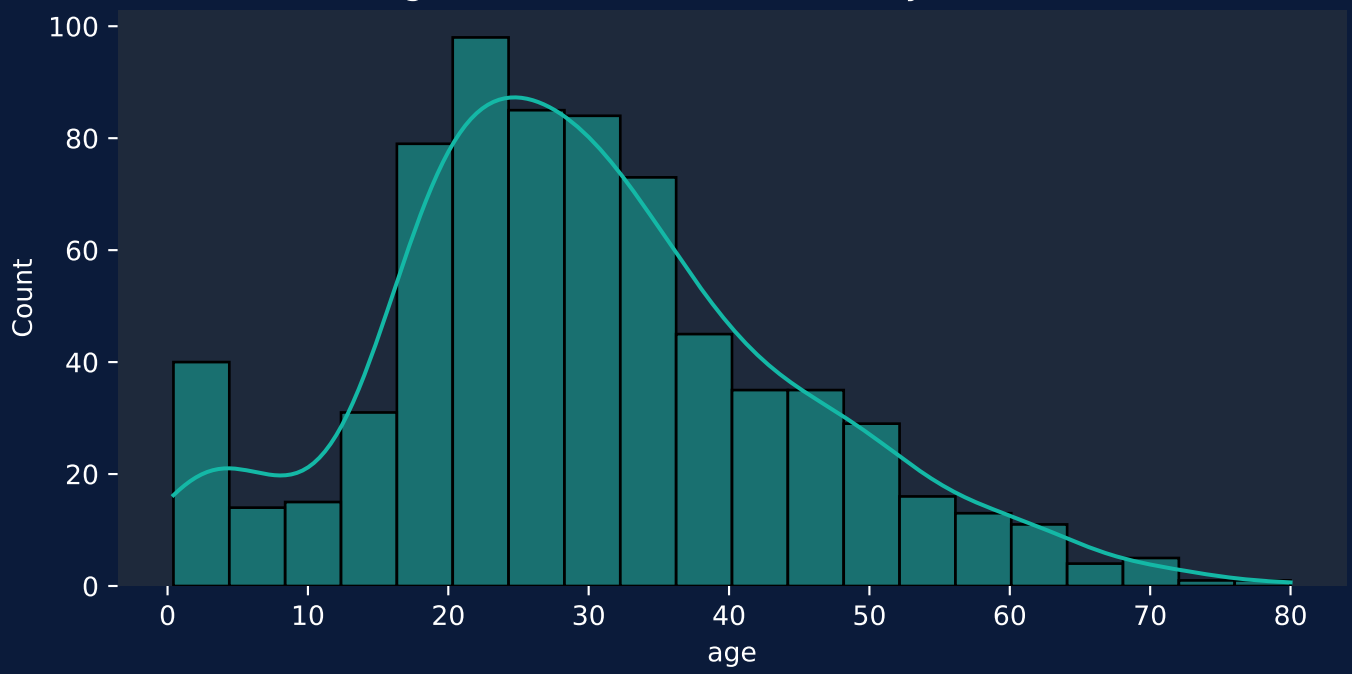
Fill `age` with median BY class (37 / 29 / 24)

107 duplicate rows → kept (different people)

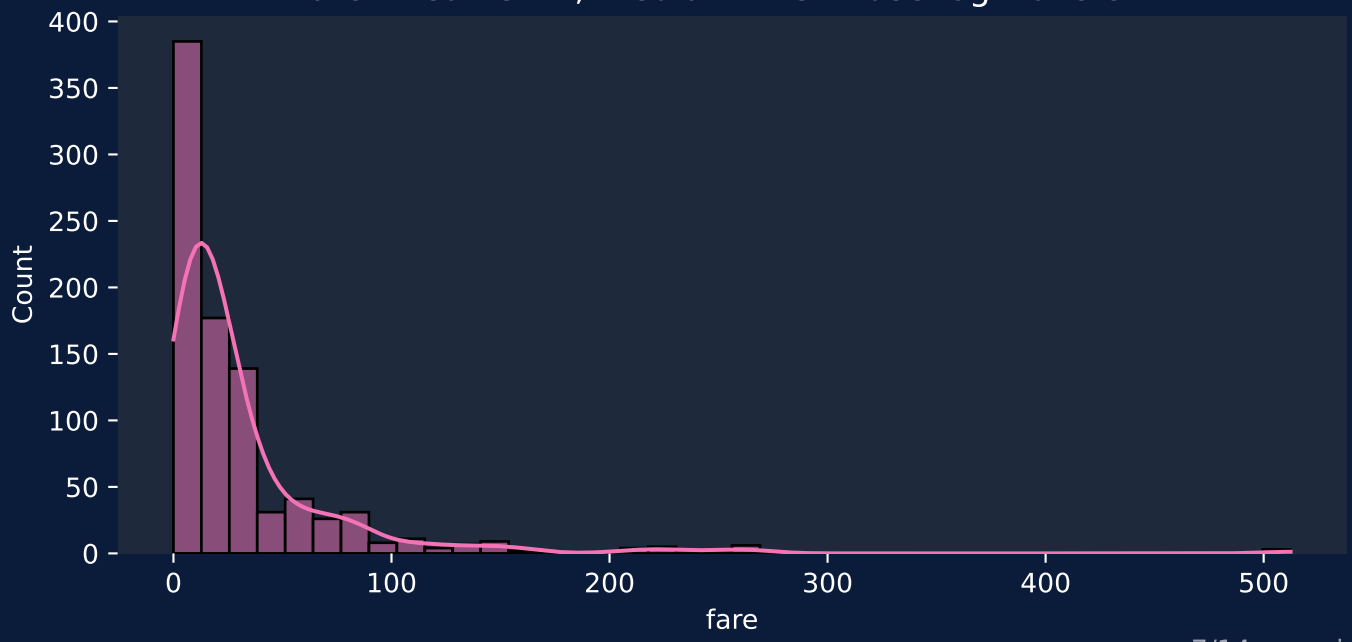
Univariate Analysis

Fare is heavily right-skewed (skew 4.79)

Age: mean 29.7, median 28 (symmetric)

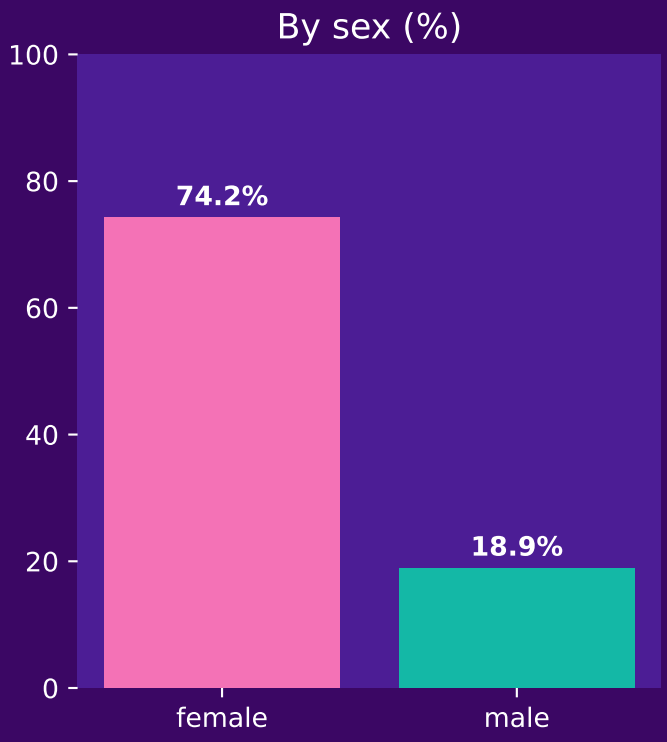


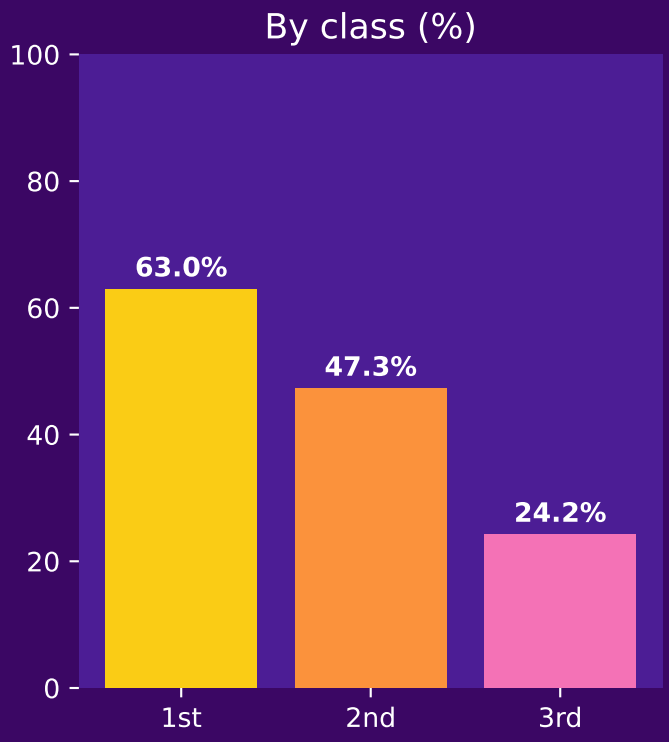
Fare: mean 32.2, median 14.5 → use log transform

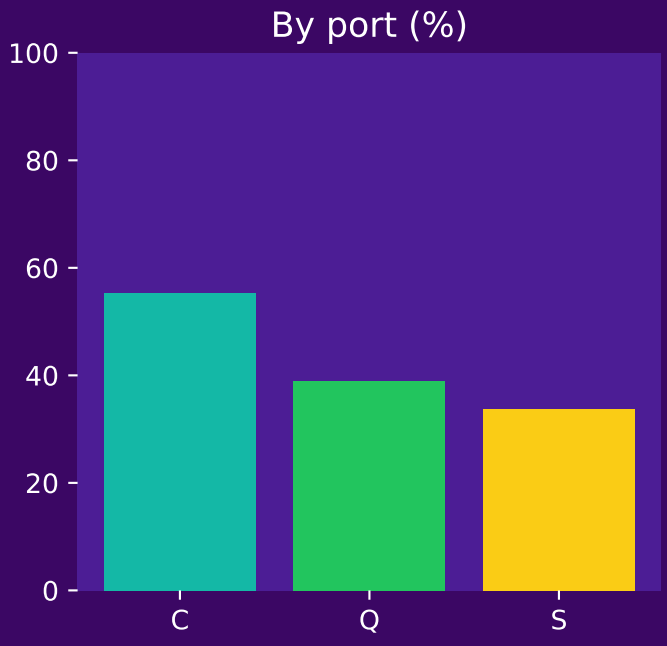


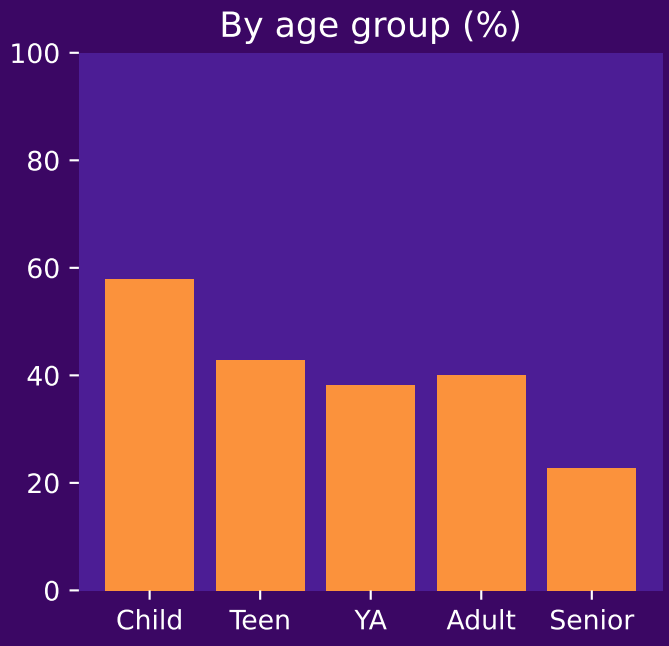
Who Survived?

Overall survival rate: 38.4%



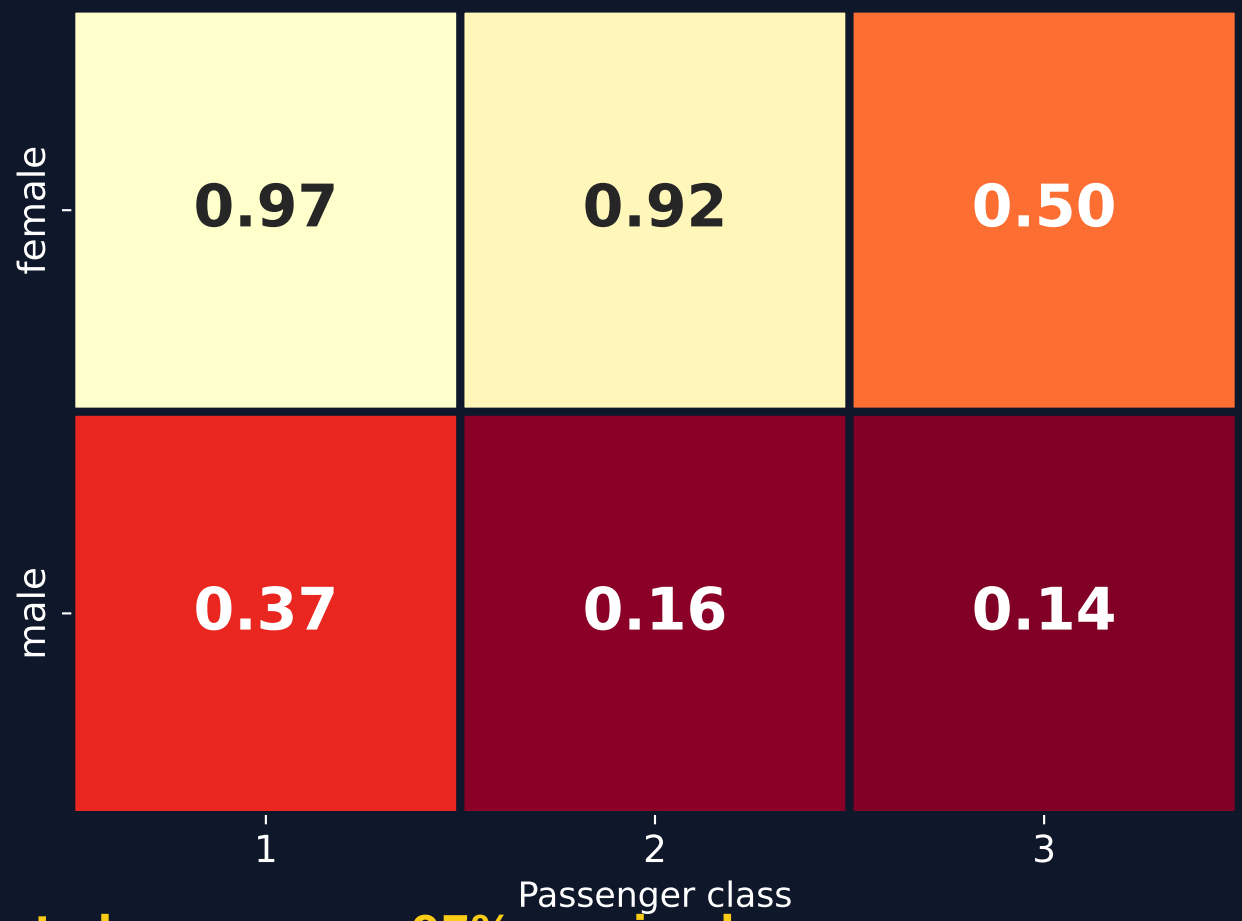






Multivariate: Sex × Class

The interaction tells the real story

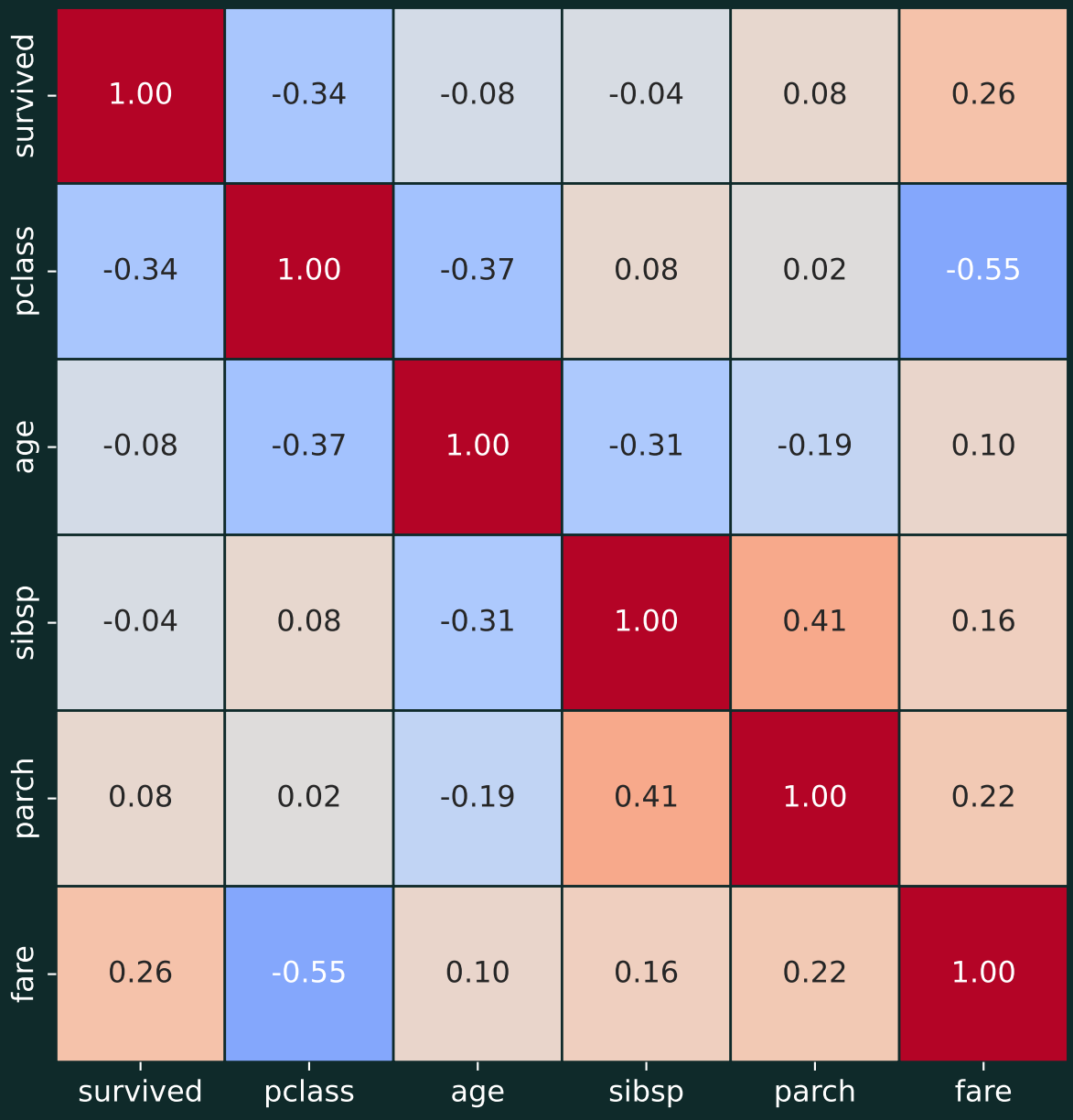


1st-class women: 97% survived

3rd-class men: only 14%

Correlation Heatmap

pclass ↔ fare = −0.55 → multicollinearity warning



Key Insights

Statistical tests confirm the visuals

Sex is the #1 predictor
$\chi^2 = 260.7$, $p \approx 1e\text{-}58$

Class #2: 63% → 47% → 24%
Fare: survivors paid £48 vs £22 ($p \approx 3e\text{-}11$)

Children & travelling with family survived more (58% / 51%)

Next: log(fare), family_size, is_alone, encode sex/embarked, handle 38/62 imbalance

Python Code (Part 1)

Copy into Google Colab and run

```
# Google Colab: paste & run
import pandas as pd, numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_theme(style="whitegrid")

df = sns.load_dataset("titanic")

# 1. Inspect
print(df.shape); df.info()
print(df.describe())

# 2. Data quality
print(df.isna().sum())
print("Duplicates:", df.duplicated().sum())

# 3. Clean
df = df.drop(columns="deck")
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])
df["age"] = df["age"].fillna(
    df.groupby("pclass")["age"].transform("median"))

# 4. Univariate
sns.histplot(df["age"], kde=True)
sns.boxplot(x=df["fare"])
print(df[["age","fare"]].skew())
```

Python Code (Part 2)

Copy into Google Colab and run

```
# 5. Bivariate
sns.barplot(x="sex", y="survived", data=df)
sns.barplot(x="pclass", y="survived",
            hue="sex", data=df)
sns.violinplot(x="survived", y="fare", data=df)

# 6. Multivariate
num = df[["survived","pclass","age",
          "sibsp","parch","fare"]]
sns.heatmap(num.corr(), annot=True,
            cmap="coolwarm", center=0)
sns.heatmap(df.pivot_table(index="sex",
    columns="pclass", values="survived"),
    annot=True, cmap="YlOrRd_r")
sns.pairplot(df[["survived","age","fare","pclass"]],
             hue="survived")

# 7. Stats test
from scipy import stats
ct = pd.crosstab(df["sex"], df["survived"])
print(stats.chi2_contingency(ct)[:2])

# 8. Feature engineering
df["family_size"] = df["sibsp"] + df["parch"] + 1
df["log_fare"] = np.log1p(df["fare"])
```

Found this useful?

Like

Repost

Follow for more Data Science

#DataScience #EDA #Python #MachineLearning #Analytics